“Alumni Connect”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

ARUNTEJ : 2520030472

SAI SHASHANK : 2520030433

YUVAN SHIVAMANI : 2520030310

Team Number: 19

Section: 6

Under the Guidance of

Dr. V. Sireesha

Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Alumni Connect"** is a record of Bonafide work of **Aruntej – 2520030472, Sai Shashank – 2520030433, Yuvan Shivamani – 2520030310** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Aruntej – 2520030472

Sai Shashank – 2520030433

Yuvan Shivamani – 2520030310

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"Alumni Connect"** is a bonafide work done and submitted by **ARUNTEJ – 2520030472, SAI SHASHANK – 2520030433, YUVAN SHIVAMANI – 2520030310** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Dr. V. Sireesha)

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr. V. Sireesha, Professor, Department of Computer Science and Engineering, for her valuable guidance, constructive suggestions and continuous encouragement throughout the development of the Alumni Connect project. Her feedback helped us to refine the problem definition, the choice of algorithms, the design of the platform and the testing process.

We also thank Dr. P. Venkateswara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the academic support and the departmental environment that were required to complete this work. We are grateful to the faculty members of the department and to our classmates for their suggestions during the demonstrations and evaluations of the project.

Finally, we thank all the members of our team for their cooperation, coding, testing, debugging and documentation efforts, which brought the project to its completed stage.

Aruntej – 2520030472

Sai Shashank – 2520030433

Yuvan Shivamani – 2520030310

Abstract

Finding a suitable mentor and learning about opportunities are difficult tasks for students, because alumni knowledge, internships, workshops and events are scattered over personal contacts and messaging groups. Alumni Connect is a web-based alumni–student mentoring platform developed as a Data Structures and Algorithms-3 (DSA-3) project to address this problem. The system has two roles, Student and Mentor, each with its own login option and dashboard. It is built with a Node.js back end that uses only the built-in http, fs, path and crypto modules, a browser front end written in HTML, CSS and vanilla JavaScript, and text files in which every line stores one JSON record, so that no database server is needed.

The platform provides registration and role-based login, a dashboard with statistics and notifications, a mentor search that ranks alumni profiles by a relevance score, a mentorship workflow in which a student sends a request that a mentor accepts or rejects, an opportunities board for internships, jobs, workshops and resources, and an events page. The relevance score combines a whole-query match, token matches and a fuzzy component based on the Levenshtein edit distance. In addition, a library of 34 hand-built algorithms covers string matching (Naive, KMP, Z, Rabin–Karp, Aho–Corasick, suffix array and suffix automaton), dynamic programming, network flow and matching, approximation, randomised algorithms and parallel-style simulations, and is exposed through 15 REST-style API endpoints.

The implementation was validated by 22 functional test cases on the running server, by running all 34 algorithms on their demonstration inputs, and by a benchmark of the exact string-matching algorithms. All designed behaviours passed, all algorithm outputs were correct, and the four exact matchers found identical matches (20, 5 and 8 occurrences of the tested patterns) on the project text. In the benchmark, Naive and KMP were the fastest, taking about 0.035 ms per search on a 3,717-character text, while the Z algorithm and Rabin–Karp were slower. The tests also showed limitations, such as plain-text storage, plain-text passwords and a fuzzy match that is limited to the beginning of the skills text, which are reported together with directions for future work.

Keywords: Alumni Connect, Mentorship Platform, Data Structures and Algorithms, String Matching, KMP, Levenshtein Distance, Dynamic Programming, Network Flow, Randomized Algorithms, Node.js, Text-File Storage.

Table of Contents

			Page No.
	Acknowledgement		
	Abstract		
	Table of Contents		
	List of Figures		
1	Introduction	1.1 Overview 1.2 Background and Motivation 1.3 Problem Statement 1.4 Objectives 1.5 Scope of the Project 1.6 Organization of the Report	1-4
2	System Requirements	2.1 Existing System and Limitations 2.2 Functional Requirements 2.3 Non-Functional Requirements 2.4 Hardware and Software Requirements	5-8
3	System Design	3.1 High-Level Architecture Diagram 3.2 Description of Each Layer 3.3 Data Model and Dataset 3.4 Mentor Search and Matching Design 3.5 Mentorship Workflow Design 3.6 Algorithm Library Design 3.7 Tools and Technologies	9-14
4	Implementation	4.1 Project Structure 4.2 Module-wise Implementation 4.3 Front-end Logic (UI Rendering, State Management) 4.4 API Integration 4.5 Algorithm Implementations (Pseudocode)	15-27
5	Results	5.1 Experimental Setup 5.2 Dataset Profile 5.3 Functional Validation 5.4 Algorithm Output Validation 5.5 Search Performance Benchmark 5.6 User Interface Results 5.7 Discussion of Results	28-36
6	Challenges & Limitations	6.1 Issues Faced During Development 6.2 Solutions Applied 6.3 Limitations of the System	37-39
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills Learned During Development	40-41

References	Books, research papers and documentation sites used	42-43

List of Figures

1.1	Overview of the Alumni Connect Platform	1
3.1	System Architecture of Alumni Connect	9
3.2	Text-File Data Model and Record Relationships	10
3.3	Mentor Search and Scoring Workflow	11
3.4	Mentorship Request Life Cycle	12
3.5	Distribution of the 34 Implemented Algorithms by Category	13
4.1	Role-Based Login Screen (Student and Mentor Options)	19
4.2	Student Dashboard	20
5.1	Number of Records in Each Runtime Data File	29
5.2	Local Benchmark: Average Search Time of the Exact Matchers	32
5.3	Find Mentors: Search Result for the Query "java"	33
5.4	Find Mentors: Search Result for the Query "python"	33
5.5	Mentorship Page of the Student	33
5.6	Opportunities Page	34
5.7	Events Page	34
5.8	Mentor Dashboard	35
5.9	Mentorship Page of the Mentor (Accepted Request)	35
6.1	No Result for the Misspelled Query "pythn"	39

1. INTRODUCTION

1.1 Overview

Alumni Connect is a full-stack web application that connects the students of an institution with experienced alumni who act as mentors. It was developed as the project component of the course Data Structures and Algorithms-3 (25CS2103E). The platform gives one organised place where a student can discover mentors by skill, industry or interest, send a mentorship request, follow its status, and view the internships, workshops and events shared by the alumni community. A mentor, in turn, can review incoming requests, accept or reject them, and publish opportunities for students. The overall idea of the platform is shown in Fig. 1.1.

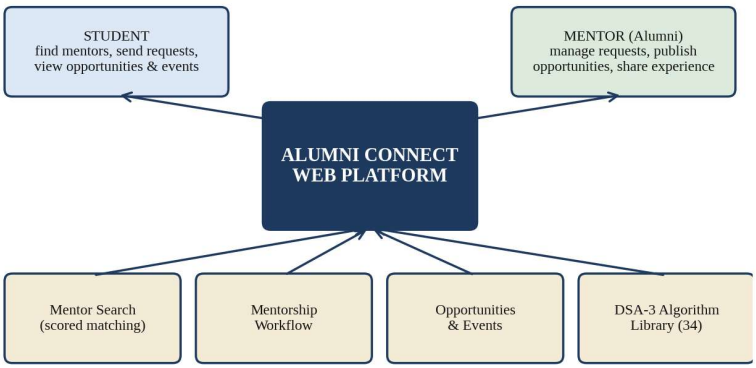


Fig. 1.1 Overview of the Alumni Connect Platform

The system is deliberately built without a database server and without a web framework. The back end is a single Node.js program (server.js) that uses only the built-in http, fs, path and crypto modules [1], and the data is stored in five plain-text files in which every line holds one record in JSON format. The front end is a single-page application written in HTML, CSS and vanilla JavaScript, which communicates with the server through JSON-based REST-style endpoints. A second source file, algorithms.js, contains thirty-four hand-written DSA-3 algorithms covering string matching, dynamic programming, network flow, approximation, randomised algorithms and parallel-style simulations. These are exposed through the /api/algorithms endpoints, and the Levenshtein edit distance is used directly in the mentor-search ranking.

The application is supplied with demonstration data: six user records (one student, four mentors and one administrator record), two mentorship requests, two opportunity posts, two events and two notifications. The project is started with the command npm start and opened in a browser at http://localhost:5000 [2].

1.2 Background and Motivation

Alumni are one of the most valuable but least organised resources of an educational institution. They have first-hand knowledge of placements, higher studies, industry skills and career growth, yet students usually reach them only through personal contacts, social-media groups or one-time events. As a result, information about internships, workshops and guidance is scattered, and a student who does not already know an alumnus has little means of finding the right person.

Finding the right mentor is essentially an information-retrieval problem. A student describes a need with a few words, such as a programming language, an industry or an area of interest, and the system must compare these words with the free-text fields of many profiles (skills, interests, industry, experience and biography) and return the most relevant people first. Exact comparison alone is not enough, because users type phrases in different forms and sometimes make spelling mistakes. The classical tools for such problems [3], [4] are pattern-matching algorithms such as Knuth–Morris–Pratt (KMP) [5] and Rabin–Karp [6], multi-pattern matching such as Aho–Corasick [7], and edit-distance measures such as the Levenshtein distance [8], [9].

The project is also educational. In many applications these techniques are hidden behind database indexes or library functions. Because Alumni Connect is a DSA-3 project, the algorithms are written explicitly in source code, tested individually and, where it is natural, connected to a real feature of the platform. The motivation is therefore two-fold: to build a useful alumni–student mentoring platform and to show how advanced data structures and algorithms fit into a practical software system.

1.3 Problem Statement

Students who need career guidance have no single, searchable place in which alumni profiles, mentorship requests, opportunities and events are kept together.

Searching manually through lists of profiles is slow, a plain keyword comparison misses relevant profiles when the wording differs slightly, and informal requests sent through messages are easily lost because their status is never tracked. Mentors, on the other hand, have no structured way to receive requests or to publish internships and workshops for students.

The problem addressed by this project is to design and implement a role-based Alumni Connect platform that (i) lets students search alumni profiles and rank them by relevance, (ii) manages the complete life cycle of a mentorship request between a student and a mentor, (iii) lets mentors share opportunities and lets all users view events and notifications, and (iv) uses explicit, hand-coded DSA-3 algorithms for the computational tasks, without depending on a database server or on external algorithm libraries.

The main challenges involved are: ranking heterogeneous free-text profiles, tolerating small differences between the query and the stored text, storing and updating data safely in plain text files, separating the Student and Mentor roles on the login screen and in the interface, and keeping the large algorithm collection modular and independently testable.

1.4 Objectives

- Develop a web platform with two explicit roles, Student and Mentor, each with its own login option and dashboard.
- Implement registration and login with validation of duplicate e-mail addresses, wrong credentials and wrong role selection.
- Provide a mentor-discovery module that searches by keyword, skill and industry and ranks the profiles by a relevance score.
- Integrate the Levenshtein edit distance as a fuzzy-matching component of the ranking.
- Implement a mentorship workflow (pending, accepted or rejected) with automatic notifications and prevention of duplicate pending requests.
- Provide pages for opportunities (internship, job, workshop, resource) and for events.

- Store all data in text files, one JSON record per line, handled by a Node.js back end without any database server.
- Implement and expose 34 hand-built DSA-3 algorithms through API endpoints.
- Validate the implementation through functional tests, algorithm output checks and a benchmark of the exact string-matching algorithms.

1.5 Scope of the Project

The scope of the project covers the Node.js server and router, text-file persistence, role-based login and registration, the dashboard, mentor search, mentorship management, opportunities, events, notifications, the algorithm library with its API, and the browser interface with an animated collapsible sidebar. Administrator records are retained in the data files for demonstration, but the login screen deliberately presents only the Student and Mentor options. The DSA-3 Algorithm Lab page was removed from the visible menu, while its API endpoints remain available [2].

The project does not include a relational or NoSQL database, e-mail delivery, real-time chat, file or resume upload, encrypted password storage or token-based sessions. It is an academic prototype intended for demonstration and algorithmic evaluation and not a production deployment.

1.6 Organization of the Report

Chapter 1 introduces the project, its motivation, the problem statement, objectives and scope. Chapter 2 reviews existing approaches and states the functional, non-functional, hardware and software requirements. Chapter 3 presents the system architecture, the data model and the design of the search, mentorship and algorithm modules. Chapter 4 explains the implementation module by module, including the front-end logic, the API and the pseudocode of the algorithms. Chapter 5 reports the functional validation, algorithm output checks, benchmark and user-interface results. Chapter 6 discusses the challenges faced and the limitations of the system. Chapter 7 concludes the report and lists future work and the skills learned. The references are given at the end.

2. SYSTEM REQUIREMENTS

2.1 Existing System and Limitations

Students and alumni are at present connected through three common means: general-purpose professional networking websites, institutional messaging groups and e-mail lists, and alumni records maintained manually by departments in documents or spreadsheets. These means are useful, but they have the following limitations.

- There is no structured mentorship workflow. A request is an informal message whose acceptance or rejection is not tracked.
- Information about opportunities and events is scattered over several channels and is easily missed.
- Keyword or filter-based search does not rank the profiles by relevance and shows no tolerance for small differences in wording.
- Generic platforms do not reflect the academic context of an institution, such as department, batch and campus events.
- For a learner of Data Structures and Algorithms, ready-made systems hide the algorithms behind database indexes and library functions, so the link between theory and practice stays invisible.

The algorithms literature describes the required techniques one at a time: linear-time pattern matching [5], hashing-based matching [6], simultaneous multi-pattern search [7], string correction through edit distance [8], [9], sequence alignment by Needleman–Wunsch [10] and Smith–Waterman [11], maximum flow by Ford–Fulkerson [12], Edmonds–Karp [13] and Dinic [14], randomised Quicksort [15], Miller–Rabin primality testing [16], reservoir sampling [17], suffix arrays [18] with linear-time LCP construction [19], bitmask dynamic programming for the travelling salesman problem [20] and suffix automata [21]. Standard textbooks [3], [4] present them as separate topics. The gap addressed by this project is the lack of a small, inspectable application in which these techniques are implemented by hand, tested together and linked to real features (profile search, fuzzy matching, student–mentor matching demonstrations), instead of being studied only as stand-alone exercises.

2.2 Functional Requirements

The functional requirements describe what the system must do. They are derived from the implemented source code and are summarised in Table 2.1.

Table 2.1 Functional Requirements

ID	Requirement	Implemented Behaviour
FR-01	Registration	A user registers with name, e-mail, password and account type (Student or Mentor). Missing fields give HTTP 400 and a duplicate e-mail gives HTTP 409. A Mentor account is stored with the internal role alumni.
FR-02	Role-based login	The login screen offers Student and Mentor options. Wrong credentials give HTTP 401 and a role mismatch gives HTTP 403. The password field is removed from every response.
FR-03	Dashboard	Shows statistics (mentor network, my mentors, opportunities, upcoming events), recommended mentors, upcoming sessions, latest opportunities and notifications for the role.
FR-04	Find mentors	Searches mentor profiles by free text, skill and industry, computes a relevance score for each mentor and returns the mentors with a positive score in descending order of score.
FR-05	Fuzzy matching	Uses the Levenshtein edit distance between the query and the skills text as one component of the score.
FR-06	Mentorship request	A student sends a request with a topic. A second pending request to the same mentor is rejected with HTTP 409.
FR-07	Mentorship management	A mentor accepts or rejects a pending request. Both roles can list the requests that involve them.
FR-08	Notifications	A notification is created when a request is sent and when its status changes. Notifications are filtered by audience (all, student, alumni).
FR-09	Opportunities	All users can view posts. Mentors can publish an internship, job, workshop or resource post with company and tags.
FR-10	Events	All users can view events. An event-creation endpoint and form are provided for the administrator role.
FR-11	Algorithm API	Lists the 34 algorithms and runs any of them on a built-in demonstration input or on user-supplied input.
FR-12	Static content and health check	Serves the front end from the public folder and provides a /api/health endpoint.

2.3 Non-Functional Requirements

The non-functional requirements describe the quality attributes of the system and are listed in Table 2.2.

Table 2.2 Non-Functional Requirements

Category	Requirement
Usability	Separate Student and Mentor login cards, a collapsible labelled sidebar, one-click search and modal forms for requests and posts, so that any feature can be demonstrated in a few steps.
Performance	Search, scoring and dashboard calls are in-memory computations over small files and complete in milliseconds. The algorithm endpoint returns the elapsed time of each run.
Modularity	Algorithms are separated in algorithms.js behind one dispatcher, the server contains the router and data helpers, and the interface is a separate static front end.
Portability	Runs on any operating system that supports Node.js 18 or later. No database server or third-party package is required.
Reliability	All API handlers are wrapped in try/catch and return JSON errors. A malformed line in a data file is read as a BAD record instead of stopping the server.
Security (basic)	User text is HTML-escaped before display, passwords are removed from responses, request bodies are limited to about 1 MB and static file paths are checked against directory traversal.
Maintainability	Small helper functions (readRecords, writeRecords, addRecord, id, json, parseBody) are reused by all handlers.
Scalability	Limited by design. Every update rewrites a complete text file, so the system suits demonstration-size data and not large user bases.

2.4 Hardware and Software Requirements

The hardware and software needed to build and run the project are given in Table 2.3.

Table 2.3 Hardware and Software Requirements

Type	Requirement	Purpose
Hardware	Laptop or desktop with keyboard and display (4 GB RAM or more recommended)	Development, running the server and browser testing.
Operating System	Windows, Linux or macOS	Run the server from a terminal such as PowerShell.
Runtime	Node.js 18 or later (declared in package.json)	Executes server.js and algorithms.js.

Type	Requirement	Purpose
Package manager	npm (used only for the npm start script)	Starts the application. There are no external dependencies.
Browser	Modern Chrome, Edge, Firefox or Safari	Opens the single-page interface at http://localhost:5000.
Editor / IDE	Visual Studio Code or any text editor	Reading and modifying the source.
Storage	Read and write access to the data folder	Holds users.txt, mentorship.txt, posts.txt, events.txt and notifications.txt.
Network	Localhost only. The port is 5000 and can be changed with the PORT environment variable	Communication between the browser and the server. Internet access is needed only for the web fonts.

3. SYSTEM DESIGN

3.1 High-Level Architecture Diagram

Alumni Connect follows a lightweight four-layer architecture, shown in Fig. 3.1. The browser sends HTTP requests to the Node.js server. The router in server.js interprets each request, uses the data-access helpers to read or write the text files, and calls the algorithm layer whenever a computation is needed. The result is returned to the browser as JSON and rendered by the front-end code.

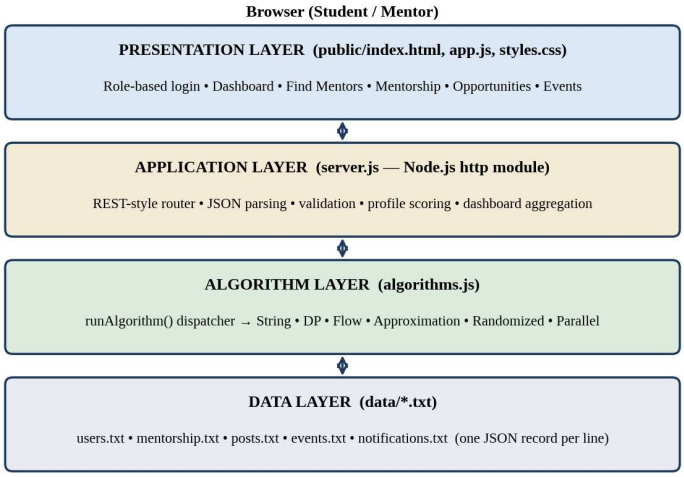


Fig. 3.1 System Architecture of Alumni Connect

3.2 Description of Each Layer

Presentation layer. It consists of index.html, app.js and styles.css. index.html contains only a root container and a toast element; app.js builds every screen (login, registration, sidebar, topbar and the five content pages) from JavaScript template strings. styles.css gives the responsive dashboard design with the animated sidebar.

Application layer. The single router function in server.js handles every request. It separates API calls (paths beginning with /api/) from static files, parses JSON bodies, validates input, applies the rules for roles and duplicates, creates identifiers and notifications, and builds the dashboard data for a user.

Algorithm layer. algorithms.js exports one function, runAlgorithm(name, input), which dispatches to 34 hand-coded algorithms. The server uses it in two

ways: the Levenshtein distance is called while scoring mentor profiles, and the /api/algorithms/run endpoint runs any algorithm on demonstration or custom input.

Data layer. Five text files in the data folder store the records. Each non-empty line is one JSON object. The server reads a whole file into memory, filters or modifies the records and, for an update, writes the whole file back.

3.3 Data Model and Dataset

The data model contains five record types. Their relationships are shown in Fig. 3.2. The user record is the principal object: mentorship requests refer to a student and a mentor through studentId and alumniId, and opportunity posts refer to their author through authorId. The fields of the user record are given in Table 3.1 and the other files in Table 3.2.

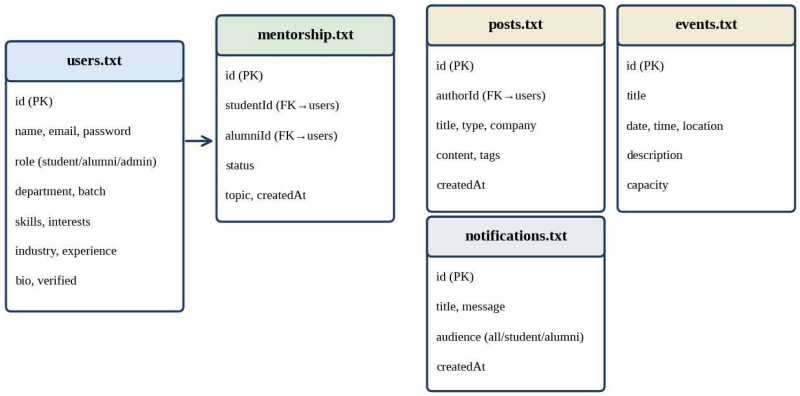


Fig. 3.2 Text-File Data Model and Record Relationships

Table 3.1 Fields of the User Record (users.txt)

Field	Meaning	Used By
id	Unique identifier such as U001	Login, mentorship, dashboard
name, email, password	Identity and credentials of the user	Login, registration, display
role	student, alumni (shown as Mentor) or admin	Role checks, mentor list
department, batch	Academic background	Profile text for search
skills, interests	Comma-separated skills and interests	Search and fuzzy matching, profile tags
industry, experience	Professional domain and years of experience	Search, mentor cards

Field	Meaning	Used By
bio, verified	Short description and verification flag	Search, profile display

Table 3.2 Fields of the Other Data Files

File	Fields
mentorship.txt	id, studentId, alumniId, status (pending, accepted or rejected), topic, createdAt
posts.txt	id, authorId, title, type, company, content, tags, createdAt
events.txt	id, title, date, time, location, description, capacity
notifications.txt	id, title, message, audience (all, student or alumni), createdAt

The supplied dataset is small but deliberately varied. It contains one student, four mentors from the CSE, ECE and IT departments with different industries (Technology, Electronics and Analytics) and one administrator record, so that search queries such as java, python, data or ECE return different results.

3.4 Mentor Search and Matching Design

The mentor search combines exact and approximate matching into one relevance score. The search workflow is shown in Fig. 3.3. For the request /api/alumni, the server keeps only the users whose role is alumni, joins the three inputs (query, skill and industry) into one search string, and calls profileScore() for every mentor.

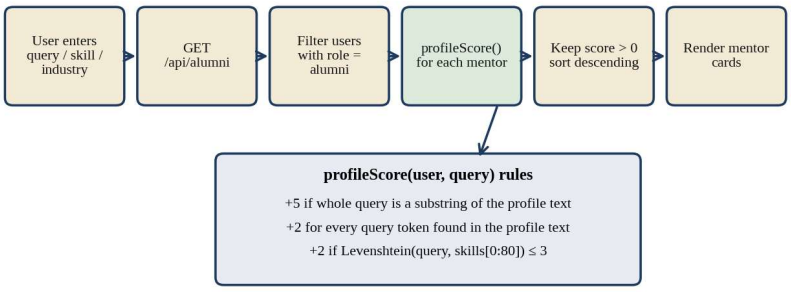


Fig. 3.3 Mentor Search and Scoring Workflow

The profile text is the lower-case concatenation of name, department, batch, skills, interests, industry, experience and bio. The score of a mentor is the sum of three parts: 5 points if the complete query is a substring of the profile text, 2 points for every query token (separated by spaces or commas) that occurs in the profile

text, and 2 further points if the Levenshtein distance between the query and the first 80 characters of the skills text is at most 3. Only mentors with a score greater than 0 are returned, ordered from the highest to the lowest score. If no query, skill or industry is entered, all mentors are returned.

For example, for the query java the profile of Priya Sharma contains the whole query (+5) and the token java (+2), giving a score of 7. For the query Java Spring the complete phrase is not a substring of the stored text, because the skills are stored as Java, Spring Boot, so only the two tokens score and the result is 4. These values were reproduced in the tests of Chapter 5.

3.5 Mentorship Workflow Design

A mentorship request has three states: pending, accepted and rejected, as shown in Fig. 3.4. A student creates a request with a topic. The server verifies that both identifiers are present and that no pending request exists for the same student and mentor; otherwise it returns HTTP 409. When the request is stored, a notification for the audience alumni is added. The mentor later changes the status to accepted or rejected; any other value gives HTTP 400, and an unknown request gives HTTP 404. After a valid update, a notification for the audience student is added. The dashboard counts, for a student, the accepted and pending requests.

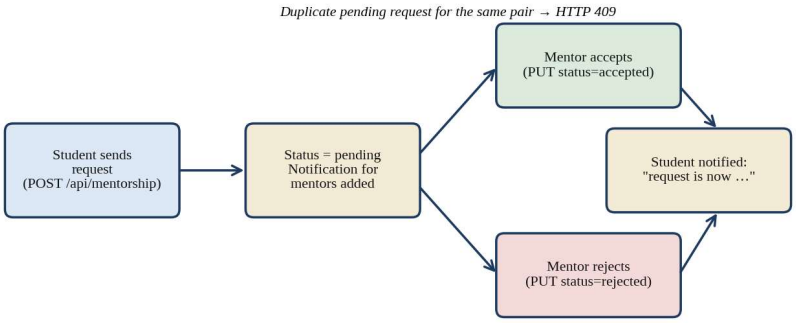


Fig. 3.4 Mentorship Request Life Cycle

3.6 Algorithm Library Design

The algorithm library groups the 34 algorithms into six categories, as shown in Fig. 3.5. The categories are string matching (7), dynamic programming and search (13), network flow and matching (6), approximation (1), randomised

algorithms (4) and parallel-style simulations (3). Each algorithm has a demonstration input in the server so that it can be run without writing any input.

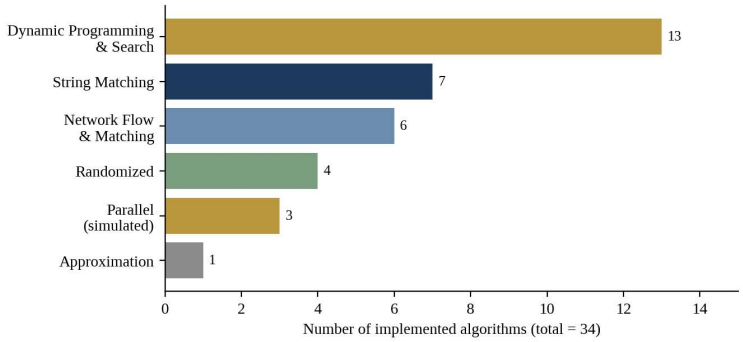


Fig. 3.5 Distribution of the 34 Implemented Algorithms by Category

Table 3.3 relates the modules of the platform to the algorithms. Only the Levenshtein distance is called by a runtime feature of the interface (mentor search). The remaining algorithms are reachable through the API and are verified in Chapter 5. The student-to-mentor matching demonstration (bipartite matching) pairs three student profiles with three mentor profiles, which illustrates how the flow-based algorithms could support automatic mentor assignment in future.

Table 3.3 Module-to-Algorithm Mapping

Module	Algorithm / Technique	Role in the System
Mentor search	Substring test, token matching, Levenshtein distance	Runtime: relevance score and fuzzy matching
Exact string search	Naive, KMP, Z, Rabin–Karp	API: pattern positions in a text
Multi-pattern search	Aho–Corasick	API: several skills searched in one pass
Text indexing	Suffix array with Kasai LCP, suffix automaton	API: suffix ordering, distinct substrings
Text similarity	Damerau–Levenshtein, Needleman–Wunsch, Smith–Waterman	API: edit distance and alignment scores
Optimisation	Knapsack, subset sum, matrix-chain, optimal BST, bitmask TSP	API: dynamic-programming demonstrations
Student–mentor matching	Bipartite matching, max-flow, min-cut, min-cost max-flow	API: assignment and flow demonstrations
Randomised and parallel	Randomised Quicksort, Miller–Rabin, reservoir sampling, hashing, reduce, prefix sum, sort	API: randomised and parallel-style demonstrations

3.7 Tools and Technologies

- **Node.js (version 18 or later)** with the built-in http, fs, path and crypto modules for the server, file storage and identifier generation [1].
- **Text files in JSON-lines format** as the database, read and written with the fs module.
- **HTML5, CSS3 and vanilla JavaScript** for the single-page front end, with the Fetch API for server calls [22] and localStorage to remember the logged-in user.
- **Google Fonts (DM Sans and Playfair Display)** referenced in styles.css; the application still works with fallback fonts when the internet is not available.
- **npm scripts** (npm start and npm run dev) to launch the server.

These tools were chosen because they keep the whole system transparent: there is no framework, ORM or database engine between the code and the algorithms, so every computational step can be read and explained.

4. IMPLEMENTATION

4.1 Project Structure

The project is organised so that the server, the algorithm library, the static front end and the data files are kept apart. Table 4.1 lists the files and their responsibilities.

Table 4.1 Source Files and Responsibilities

Path	Responsibility
server.js	Node.js HTTP server: router, data-access helpers, authentication, mentor scoring, dashboard aggregation, mentorship, posts, events, notifications and algorithm endpoints.
algorithms.js	Hand-built implementations of the 34 algorithms and the runAlgorithm() dispatcher.
public/index.html	Root page with the application container, the toast element and the script reference.
public/app.js	Front-end state, screen rendering, API calls, modal forms and event handling.
public/styles.css	Responsive styling, animated sidebar, cards, forms and toast messages.
data/users.txt	User records (student, mentors and administrator).
data/mentorship.txt	Mentorship requests and their status.
data/posts.txt	Opportunity posts (internships, jobs, workshops, resources).
data/events.txt	Campus and alumni events.
data/notifications.txt	Role-based notifications.
package.json	Project metadata, the npm start script and the Node.js version requirement (18 or later).
README.md, design-reference.png	Run instructions, demo accounts, list of front-end changes and the dashboard design reference.

4.2 Module-wise Implementation

This section explains the modules of the back end in the order in which a request uses them. The pseudocode follows the structure of the source code.

4.2.1 Data Access Module

All five data files are handled by three small functions. readRecords() returns the list of records of a file; a line that is not valid JSON is returned as a record with

the identifier BAD-i so that one damaged line does not stop the server. writeRecords() rewrites the whole file, and addRecord() appends one record. New identifiers are made by id(prefix), which adds eight random hexadecimal characters generated with the crypto module to a prefix such as U, M, P, E or N.

Algorithm 1: Data Access

```
FUNCTION readRecords(name)
    IF file data/name.txt does not exist THEN RETURN empty list
    FOR each non-empty line of the file
        TRY record <- JSON.parse(line)
        ON ERROR record <- { id: "BAD-" + index, raw: line }
    RETURN list of records
FUNCTION addRecord(name, record)
    records <- readRecords(name); APPEND record
    writeRecords(name, records)   // one JSON object per line
    RETURN record
FUNCTION id(prefix) RETURN prefix + uppercase hex of 4 random bytes
```

4.2.2 Authentication Module

Login receives e-mail, password and the role chosen on the login card. The Mentor option is translated to the stored role alumni. The server returns the public user (without the password) and the dashboard data. Registration validates the required fields, rejects a duplicate e-mail address and stores the new user with default department CSE, batch 2026 and verified set to false.

Algorithm 2: Login and Registration

```
FUNCTION login(email, password, chosenRole)
    requiredRole <- "alumni" IF chosenRole = "mentor" ELSE "student"
    user <- findUser(email)
    IF user is missing OR user.password != password THEN RETURN 401
    "Invalid email or password."
    IF user.role != requiredRole THEN RETURN 403 "Please choose the
    correct login option."
    RETURN 200 { userPublic(user), dashboard(user.id) }
FUNCTION register(body)
    IF name, email or password is missing THEN RETURN 400
    IF an existing user has the same e-mail (case-insensitive) THEN RETURN
    409
    role <- "alumni" IF body.role is "mentor" or "alumni" ELSE "student"
    user <- new record with id("U"), defaults and verified = false
    addRecord("users", user)
    RETURN 201 { userPublic(user), dashboard(user.id) }
```

4.2.3 Mentor Search and Scoring Module

The /api/alumni handler selects the users with role alumni, scores each of them with profileScore() and returns those with a positive score. The scoring function is the only place where an algorithm from algorithms.js is called during normal use of the interface, as explained in Section 3.4.

Algorithm 3: Mentor Search with Fuzzy Scoring

```
FUNCTION profileScore(user, query)
    q <- lowercase(trim(query)); IF q is empty THEN RETURN 0
    text <- lowercase(name + department + batch + skills + interests + industry
    + experience + bio)
    score <- 0
    IF q is a substring of text THEN score <- score + 5
    FOR each token of q split by spaces or commas
        IF token is a substring of text THEN score <- score + 2
    d <- Levenshtein(q, first 80 characters of lowercase(user.skills))
    IF d <= 3 THEN score <- score + 2
    RETURN score
FUNCTION searchMentors(q, skill, industry)
    mentors <- users WHERE role = "alumni"
    query <- join the non-empty values of q, skill, industry with a space
    FOR each mentor m: m.score <- profileScore(m, query)
    IF q, skill and industry are all empty THEN RETURN all mentors
    RETURN mentors WHERE score > 0, sorted by score in descending order
```

4.2.4 Mentorship and Notification Module

A request stores the student, the mentor, the topic, the status and the creation time. Each change also writes a notification record, which the dashboard and the notification list later read according to the audience.

Algorithm 4: Mentorship Request and Status Update

```
FUNCTION createRequest(studentId, alumniId, topic)
    IF studentId or alumniId is missing THEN RETURN 400
    IF a pending request exists for the same studentId and alumniId THEN
    RETURN 409
    m <- { id("M"), studentId, alumniId, status = "pending", topic, createdAt =
    now }
    addRecord("mentorship", m)
    addRecord("notifications", { title = "New mentorship request", audience =
    "alumni" })
    RETURN 201 m
FUNCTION updateRequest(requestId, status)
    m <- request with id = requestId; IF none THEN RETURN 404
    IF status is not "accepted" or "rejected" THEN RETURN 400
    m.status <- status; writeRecords("mentorship", all requests)
    addRecord("notifications", { title = "Mentorship update", audience =
    "student" })
    RETURN 200 m
```

4.2.5 Opportunities, Events and Dashboard Modules

Opportunity posts and events are stored as simple records. A post has a type (internship, job, workshop or resource, with opportunity as the default), a company or organiser, a content text and comma-separated tags. An event has a date, time, location, description and capacity (default 50). The dashboard handler gathers the data needed by the home page of a user in one response.

Algorithm 5: Dashboard Aggregation

```
FUNCTION dashboard(userId)
    user <- findUser(userId); role <- user.role
    mentorCount <- number of users with role "alumni"
    accepted <- requests of this student with status "accepted"
    pending <- requests of this student with status "pending"
    posts <- last 4 posts (newest first) of type internship or workshop (all types
    for admin)
    notes <- last 5 notifications (newest first) whose audience is "all" or equals
    role
    RETURN { user, stats = { mentorCount, accepted, pending, totalPosts,
    totalEvents },
                upcoming = last 4 events, posts, notes }
```

4.2.6 Algorithm Service Module

The algorithm endpoints let any algorithm be run from a client. If the request has no input, the server uses the built-in demonstration input for that algorithm. The time taken is measured and returned together with the result.

Algorithm 6: Algorithm Run Endpoint

```
FUNCTION runEndpoint(algorithm, input)
    IF input is empty THEN input <- demonstrationInput[algorithm]
    IF input is still empty THEN RETURN 400 "No demo/input available"
    start <- current time
    result <- runAlgorithm(algorithm, input)   // switch on algorithm name
    RETURN 200 { algorithm, result, input, elapsedMs = current time - start }
```

4.3 Front-end Logic (UI Rendering, State Management)

The interface is a single-page application. All screens are produced by JavaScript functions that return template strings, and the page is updated by assigning the result to the container. The application state is held in one object, state, with the fields user, dashboard, view, navOpen and loginRole. A small api() wrapper calls fetch(), sends and parses JSON and throws an Error with the message returned by the server, which is then shown as a toast message.

UI rendering. renderAuth() draws the login and registration card. renderLogin() shows the Student and Mentor role cards (Fig. 4.1) and fills in the demo e-mail and password of the selected role. After a successful login, renderApp() draws the shell (sidebar, top bar and page area) and loadView() fetches

fresh dashboard data and renders the selected view: dashboardView, alumniView, mentorshipView, opportunitiesView or eventsView. After each render, bindView() starts the data loading that the view needs, for example searchAlumni() or loadMentorship().

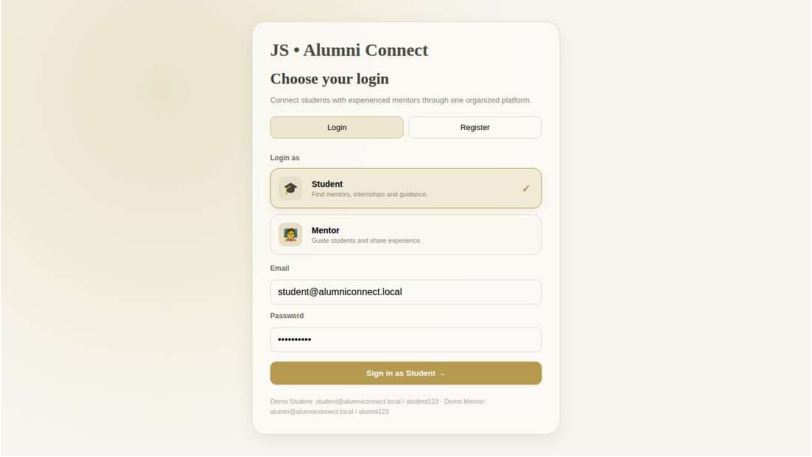


Fig. 4.1 Role-Based Login Screen (Student and Mentor Options)

State management. The selected sidebar entry is stored in state.view and the sidebar open or closed condition in state.navOpen, which toggles the CSS class nav-open to animate the sidebar. The logged-in user is saved in the browser through localStorage under the key alumniUser, so that a page reload restores the session; logout() removes it and shows the login card again.

Role-dependent behaviour. A student sees the Request Mentorship and Connect buttons and the Find a Mentor shortcut. A mentor sees a Mentor badge instead, the Accept and Reject buttons for pending requests, and the Publish Opportunity button. The Create Event button is shown only to the administrator role. The dashboard of the student is shown in Fig. 4.2.

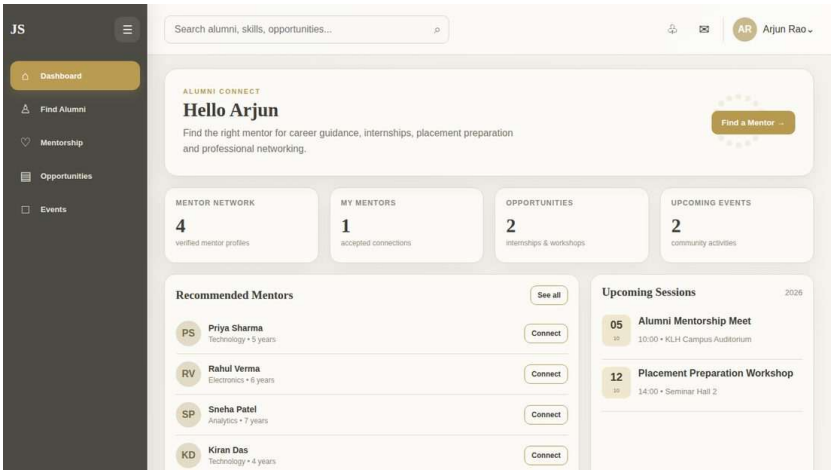


Fig. 4.2 Student Dashboard

Safe rendering and shortcuts. Every value that comes from the data files is passed through the esc() function, which replaces the characters &, <, >, " and a single quote, before it is placed in HTML. Pressing Enter in the global search box opens Find Alumni and runs the search with the entered text, and the notification button scrolls to the notification list of the dashboard.

4.4 API Integration

The browser and the server exchange JSON through the endpoints in Table 4.2. All responses carry the headers Content-Type: application/json, Access-Control-Allow-Origin: * and Cache-Control: no-store, and the server answers OPTIONS requests with status 204. The front-end functions call these endpoints with fetch(): for example, searchAlumni() calls GET /api/alumni with the query, skill and industry encoded in the URL, and updateMentor() calls PUT /api/mentorship/{id} with the new status.

Table 4.2 API Endpoints

Method	Endpoint	Purpose	Main Status Codes
GET	/api/health	Service check	200
POST	/api/register	Create a user account	201, 400, 409
POST	/api/login	Role-based login	200, 401, 403
GET	/api/dashboard?userId=	Dashboard data of a user	200, 404
GET	/api/alumni?q=&skill=&industry=	Search and rank mentors	200

Method	Endpoint	Purpose	Main Status Codes
GET	/api/mentorship?userId=	Requests of a student or mentor	200
POST	/api/mentorship	Send a mentorship request	201, 400, 409
PUT	/api/mentorship/:id	Accept or reject a request	200, 400, 404
GET	/api/posts	List opportunities (newest first)	200
POST	/api/posts	Publish an opportunity	201
GET	/api/events	List events	200
POST	/api/events	Create an event	201
GET	/api/notifications?role=	Notifications for a role	200
GET	/api/algorithms	List the 34 algorithms	200
POST	/api/algorithms/run	Run an algorithm on demo or custom input	200, 400, 500

4.5 Algorithm Implementations (Pseudocode)

This section gives the pseudocode of the algorithms in algorithms.js. The algorithms were written by hand, without external DSA libraries. The text variable has length n, the pattern has length m and the symbol T denotes the target value of a subset-sum problem. Table 4.3 at the end of the section summarises the time and space complexity of all 34 algorithms.

4.5.1 Exact String Matching

The four single-pattern matchers return the list of starting positions of the pattern in the text [3], [4]. The Naive matcher compares the pattern at every alignment and needs no preprocessing. KMP builds the longest-prefix-suffix (LPS) table of the pattern so that, after a mismatch, the pattern moves forward without re-reading text characters [5]. The Z algorithm computes the Z-array of pattern + "$" + text, and a Z value not smaller than the pattern length marks a match. Rabin–Karp compares a rolling hash of the pattern with the hash of each text window and verifies a hash match character by character [6]; the implementation uses base 31 and modulus 1,000,000,007.

Algorithm 7: Naive Pattern Matching

```
FOR i <- 0 TO n - m
    j <- 0
    WHILE j < m AND text[i + j] = pattern[j]: j <- j + 1
    IF j = m THEN record i
```

Algorithm 8: KMP Pattern Matching

```
FUNCTION buildLPS(p)
    lps[0] <- 0; len <- 0; i <- 1
    WHILE i < m
        IF p[i] = p[len] THEN len <- len + 1; lps[i] <- len; i <- i + 1
        ELSE IF len > 0 THEN len <- lps[len - 1]
        ELSE lps[i] <- 0; i <- i + 1
FUNCTION kmp(text, p)
    lps <- buildLPS(p); i <- 0; j <- 0
    WHILE i < n
        IF text[i] = p[j] THEN i <- i + 1; j <- j + 1
            IF j = m THEN record i - j; j <- lps[j - 1]
        ELSE IF j > 0 THEN j <- lps[j - 1]
        ELSE i <- i + 1
```

Algorithm 9: Z Pattern Matching

```
s <- pattern + "$" + text; compute Z[1..|s|-1] with window [l, r]
    FOR i <- 1 TO |s| - 1
        IF i <= r THEN Z[i] <- MIN(r - i + 1, Z[i - l])
        WHILE i + Z[i] < |s| AND s[Z[i]] = s[i + Z[i]]: Z[i] <- Z[i] + 1
        IF i + Z[i] - 1 > r THEN l <- i; r <- i + Z[i] - 1
FOR i <- m + 1 TO |s| - 1
    IF Z[i] >= m THEN record i - (m + 1)
```

Algorithm 10: Rabin-Karp Pattern Matching

```
compute hash(pattern) and hash(text[0..m-1]) with base 31, modulus
1000000007
pow <- 31^(m-1) mod modulus
FOR i <- 0 TO n - m
    IF patternHash = windowHash AND text[i..i+m-1] = pattern THEN record i
    IF i < n - m THEN
        windowHash <- (windowHash - text[i] * pow) mod modulus   // make
        non-negative
        windowHash <- (windowHash * 31 + text[i + m]) mod modulus
```

4.5.2 Multi-Pattern Matching and Suffix Structures

Aho–Corasick builds a trie of all patterns, adds failure links by a breadth-first traversal and then scans the text once, reporting every pattern that ends at each position [7]. In the platform, this models searching several skills in the profile text of a mentor in a single pass. The suffix array lists the starting positions of all suffixes in sorted order [18]; the implementation sorts the suffixes by comparison, and Kasai's algorithm then derives the longest-common-prefix (LCP) array in linear time [19]. The suffix automaton is built incrementally and gives the number of states and of distinct substrings of a text [21].

Algorithm 11: Aho-Corasick Multi-Pattern Matching

```
build trie: FOR each pattern k, insert its characters; store k in out[last node]
FOR each child u of the root: fail[u] <- root; enqueue u
WHILE queue is not empty
    v <- dequeue
    FOR each edge (v, c, u)
        enqueue u; f <- fail[v]
        WHILE f is not root AND f has no edge c: f <- fail[f]
        fail[u] <- child(f, c) IF it exists and differs from u ELSE root
        out[u] <- out[u] + out[fail[u]]
v <- root
FOR each text position i with character c
    WHILE v is not root AND v has no edge c: v <- fail[v]
    IF v has edge c THEN v <- child(v, c)
    FOR each pattern k in out[v]: record (pattern k, start i - |pattern k| + 1)
```

Algorithm 12: Suffix Array with Kasai LCP

```
sa <- positions 0..n-1 sorted by comparing the suffixes starting at those positions
FOR i <- 0 TO n - 1: rank[sa[i]] <- i
k <- 0
FOR i <- 0 TO n - 1
    IF rank[i] = n - 1 THEN k <- 0; CONTINUE
    j <- sa[rank[i] + 1]
    WHILE i + k < n AND j + k < n AND s[i + k] = s[j + k]: k <- k + 1
    lcp[rank[i]] <- k
    IF k > 0 THEN k <- k - 1
```

4.5.3 Edit Distance and Sequence Alignment

The Levenshtein distance is the minimum number of insertions, deletions and substitutions that change one string into another [8], [9]. It is computed with a two-dimensional table and is the algorithm used by the mentor search. Damerau–Levenshtein also counts the transposition of two adjacent characters as a single operation, so that alumni and almuni are at distance 1. Needleman–Wunsch [10] computes a global alignment score and Smith–Waterman [11] the best local alignment score, with the scores match = 2, mismatch = -1, gap = -2 and match = 3, mismatch = -3, gap = -2 respectively.

Algorithm 13: Levenshtein Distance

```
create d[0..|a|][0..|b|]
FOR i <- 0 TO |a|: d[i][0] <- i
FOR j <- 0 TO |b|: d[0][j] <- j
FOR i <- 1 TO |a|
    FOR j <- 1 TO |b|
        IF a[i-1] = b[j-1] THEN d[i][j] <- d[i-1][j-1]
        ELSE d[i][j] <- 1 + MIN(d[i-1][j], d[i][j-1], d[i-1][j-1])
RETURN d[|a|][|b|]
```

Algorithm 14: Needleman-Wunsch and Smith-Waterman

```
Needleman-Wunsch: d[i][0] <- i * gap; d[0][j] <- j * gap
    d[i][j] <- MAX(d[i-1][j-1] + (match OR mismatch), d[i-1][j] + gap, d[i][j-1]
    + gap)
    RETURN d[|a|][|b|]
Smith-Waterman: d[i][0] <- 0; d[0][j] <- 0; best <- 0
    d[i][j] <- MAX(0, d[i-1][j-1] + (match OR mismatch), d[i-1][j] + gap, d[i][j-
    1] + gap)
    best <- MAX(best, d[i][j]); RETURN best
```

4.5.4 Dynamic Programming and Search

The 0/1 knapsack and subset-sum problems use a one-dimensional table that is filled from the largest capacity downward, so that each item is used at most once. Matrix-chain multiplication and the optimal binary search tree fill an interval table by increasing interval length. The travelling-salesman problem is solved by Held–Karp bitmask dynamic programming [20], in which the state is the set of visited cities and the last city. The Hamiltonian path is found by backtracking. The Tree/Subset DP entry reuses the subset-sum recurrence, the tree diameter uses two

breadth-first searches, and the sum-over-subsets (SOS) DP adds the value of every submask to its mask.

Algorithm 15: 0/1 Knapsack and Subset Sum

```
dp[0..C] <- 0                          // subset sum: dp[0] <- TRUE, others FALSE
FOR each item i with weight w[i] and value v[i]
    FOR c <- C DOWNTO w[i]
        dp[c] <- MAX(dp[c], dp[c - w[i]] + v[i])    // subset sum: dp[c] <- dp[c]
        OR dp[c - w[i]]
RETURN dp[C]
```

Algorithm 16: Matrix-Chain Multiplication

```
FOR len <- 2 TO n
    FOR i <- 0 TO n - len
        j <- i + len - 1; m[i][j] <- infinity
        FOR k <- i TO j - 1
            m[i][j] <- MIN(m[i][j], m[i][k] + m[k+1][j] + p[i] * p[k+1] * p[j+1])
RETURN m[0][n-1]
```

Algorithm 17: Bitmask TSP (Held-Karp)

```
dp[mask][u] <- infinity; dp[1][0] <- 0
FOR mask <- 1 TO 2^n - 1
    FOR each u in mask with dp[mask][u] < infinity
        FOR each v not in mask
            dp[mask OR (1 << v)][v] <- MIN(dp[mask OR (1 << v)][v],
            dp[mask][u] + dist[u][v])
RETURN MIN over u of dp[2^n - 1][u] + dist[u][0]
```

4.5.5 Network Flow and Matching

The maximum flow is found by repeatedly searching a path with spare capacity from the source to the sink and pushing the bottleneck amount along it [12], [13]. The Ford–Fulkerson entry and the Edmonds–Karp entry use the same breadth-first augmenting-path implementation, so they return the same flow. Dinic's algorithm builds a level graph by breadth-first search and then sends blocking flows by depth-first search [14]. The minimum cut is read from the residual graph: the vertices still reachable from the source form the source side. Bipartite matching assigns each left vertex (a student) to a free right vertex (a mentor) by depth-first search with re-assignment, and the min-cost max-flow repeatedly augments along a cheapest path.

Algorithm 18: Edmonds-Karp Maximum Flow and Min-Cut

```
r <- copy of capacity matrix; flow <- 0
LOOP
    BFS from source over edges with r[u][v] > 0, storing parent[v]
    IF sink was not reached THEN EXIT LOOP
    add <- MIN of r[parent[v]][v] along the path from sink back to source
    FOR each edge (u, v) on the path: r[u][v] <- r[u][v] - add; r[v][u] <- r[v][u]
    + add
    flow <- flow + add
min-cut source side <- vertices reachable from source in the final residual graph
RETURN flow
```

Algorithm 19: Bipartite Matching (Student-Mentor Assignment)

```
match[v] <- -1 for every right vertex v; count <- 0
FUNCTION try(u, seen)
    FOR each right vertex v adjacent to u
        IF seen[v] THEN CONTINUE
        seen[v] <- TRUE
        IF match[v] = -1 OR try(match[v], seen) THEN match[v] <- u;
        RETURN TRUE
    RETURN FALSE
FOR each left vertex u: IF try(u, new seen array) THEN count <- count + 1
RETURN count and the matched pairs
```

Algorithm 20: Vertex Cover 2-Approximation

```
cover <- empty set
FOR each edge (u, v)
    IF neither u nor v is covered THEN add u and v to cover
RETURN cover   // at most twice the size of an optimal vertex cover
```

4.5.6 Randomised and Parallel-Style Algorithms

Randomised Quicksort picks a random pivot, which makes the expected running time O(n log n) for every input order [15]. Miller–Rabin tests primality by checking a set of fixed bases and is exact for the numbers used in the demonstration [16]. Reservoir sampling keeps k items chosen uniformly from a sequence of unknown length in one pass [17]. The universal-style hash multiplies and adds the characters of a key modulo a prime and then reduces the result modulo the table size. The parallel reduce, prefix sum and sort are simulations that run sequentially in a single Node.js process; the sort uses the built-in array sort.

Algorithm 21: Randomised Quicksort

```
FUNCTION qs(l, r)
    IF l >= r THEN RETURN
    p <- random integer in [l, r]; SWAP arr[p] and arr[r]; x <- arr[r]; i <- l
    FOR j <- l TO r - 1
        IF arr[j] <= x THEN SWAP arr[i] and arr[j]; i <- i + 1
    SWAP arr[i] and arr[r]
    qs(l, i - 1); qs(i + 1, r)
```

Algorithm 22: Miller-Rabin and Reservoir Sampling

```
Miller-Rabin(n): IF n < 2 THEN RETURN FALSE; test small primes 2..37 by direct division
    write n - 1 = 2^s * d with d odd
    FOR each base a in {2, 3, 5, 7, 11, 13, 17}
        x <- a^d mod n; IF x = 1 OR x = n - 1 THEN CONTINUE
        repeat s - 1 times: x <- x^2 mod n; IF x = n - 1 THEN pass
        IF no pass THEN RETURN FALSE
    RETURN TRUE
Reservoir(arr, k): r <- first k items
    FOR i <- k TO |arr| - 1
        j <- random integer in [0, i]; IF j < k THEN r[j] <- arr[i]
    RETURN r
```

Table 4.3 gives the complexity of every implemented algorithm. For the string algorithms n is the text length and m the pattern length; for the dynamic-programming algorithms m and n are the lengths of the two inputs or the number of items; V and E are the numbers of vertices and edges.

Table 4.3 Complexity of the Implemented Algorithms

Algorithm	Time Complexity	Space
Naive search	O(nm)	O(1)
KMP	O(n + m)	O(m)
Z-function search	O(n + m)	O(n + m)
Rabin–Karp	O(n + m) average, O(nm) worst	O(1)
Aho–Corasick	O(n + total pattern length + matches)	O(total pattern length)
Suffix array (comparison sort)	O(n² log n) worst	O(n)
Kasai LCP	O(n)	O(n)
Suffix automaton	O(n) for a fixed alphabet	O(n)

Algorithm	Time Complexity	Space
Levenshtein, Damerau–Levenshtein	$O(mn)$	$O(mn)$
Needleman–Wunsch, Smith–Waterman	$O(mn)$	$O(mn)$
Matrix-chain, optimal BST	$O(n^3)$	$O(n^2)$
Bitmask TSP	$O(2^n n^2)$	$O(2^n n)$
Subset sum, Tree/Subset DP	$O(nT)$	$O(T)$
0/1 Knapsack	$O(nC)$	$O(C)$
Hamiltonian path (backtracking)	$O(n!)$ worst case	$O(n)$
Tree diameter	$O(V + E)$	$O(V)$
SOS DP	$O(N \log N)$, $N = 2^k$	$O(N)$
Ford–Fulkerson / Edmonds–Karp (BFS)	$O(VE^2)$ bound	$O(V^2)$
Min-cut from residual graph	Max-flow time + $O(V^2)$	$O(V^2)$
Dinic	$O(V^2E)$ bound	$O(V^2)$
Bipartite matching	$O(VE)$	$O(V)$
Min-cost max-flow	$O(F \cdot V^2)$, F = total flow	$O(V + E)$
Vertex cover 2-approximation	$O(E)$	$O(V)$
Randomised Quicksort	$O(n \log n)$ expected, $O(n^2)$ worst	$O(n)$ copy + $O(\log n)$ stack
Miller–Rabin	$O(k \log^3 n)$	$O(1)$
Universal-style hash	$O(\text{key length})$	$O(1)$
Reservoir sampling	$O(n)$	$O(k)$
Parallel reduce / prefix sum (simulated)	$O(n)$	$O(1)$ / $O(n)$
Parallel sort (simulated, built-in sort)	$O(n \log n)$	$O(n)$

5. RESULTS

5.1 Experimental Setup

The results combine three kinds of evidence obtained from the supplied project: (1) functional tests of the running server through its HTTP API and browser interface, (2) the output of all 34 algorithms for their built-in demonstration inputs, and (3) a local benchmark of the exact string-matching algorithms. All tests were run on the supplied source code with the original demonstration data, using Node.js v22 on a Linux system with the server on a local port. The benchmark text is the lower-case concatenation of the five data files (3,717 characters) and, for a larger test, the same text repeated 50 times (185,850 characters). Each algorithm was timed for three patterns (alumni, java and mentor); the small text used 2,000 warm-up calls followed by 5,000 timed calls, and the large text used 50 warm-up calls followed by 200 timed calls.

Because execution time depends on processor load, the JavaScript engine state and garbage collection, the timings should be read as a comparison between the algorithms and not as fixed guarantees. The functional outputs are used to confirm correctness.

5.2 Dataset Profile

The runtime dataset is summarised in Table 5.1 and Fig. 5.1. It has 6 user records, of which 4 are mentors from three departments (CSE 2, ECE 1 and IT 1) with 4 to 7 years of experience and batches from 2019 to 2022. The two mentorship records are in different states, which allows both the accepted and the pending paths of the workflow to be shown.

Table 5.1 Summary of the Runtime Dataset

Data File	Records	Details
users.txt	6	1 student, 4 mentors (Technology 2, Electronics 1, Analytics 1) and 1 administrator
mentorship.txt	2	1 accepted request and 1 pending request, both from the demo student
posts.txt	2	1 internship (Backend Engineering Internship) and 1 workshop (Analytics Career Workshop)

Data File	Records	Details
events.txt	2	Alumni Mentorship Meet (capacity 100) and Placement Preparation Workshop (capacity 80)
notifications.txt	2	1 notification for all users and 1 for students

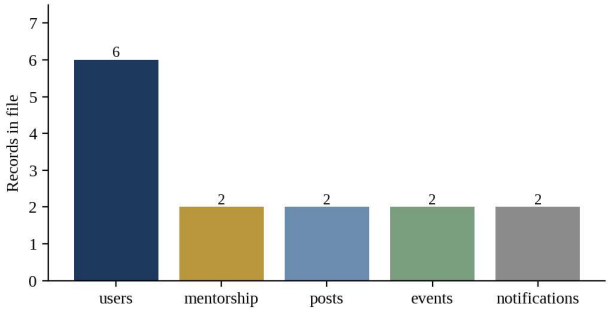


Fig. 5.1 Number of Records in Each Runtime Data File

5.3 Functional Validation

The API and the interface were tested with the cases in Table 5.2. Each case was run against the live server and the observed result is reported. All cases passed except TC-12, which is a limitation that was observed and is discussed in Section 6.3.

Table 5.2 Functional Validation Results

ID	Test Case	Observed Result	Status
TC-01	Login as Student with the demo credentials	User returned without password; dashboard stats: 4 mentors, 1 accepted, 1 pending, 2 opportunities, 2 events	PASS
TC-02	Login with a wrong password	HTTP 401, "Invalid email or password."	PASS
TC-03	Student credentials with the Mentor option	HTTP 403, message that the account is registered as Student	PASS
TC-04	Register a new student	HTTP 201, new identifier created, password not present in the response	PASS
TC-05	Register with an existing e-mail	HTTP 409, "Email already registered."	PASS
TC-06	Register without a password	HTTP 400, "Name, email and password are required."	PASS
TC-07	Register with account type Mentor	HTTP 201, stored with role alumni	PASS
TC-08	Search query java	Priya Sharma only, score 7	PASS
TC-09	Search query python	Rahul Verma (7) and Sneha Patel (7)	PASS
TC-10	Search query Java Spring	Priya Sharma, score 4 (two token matches)	PASS

ID	Test Case	Observed Result	Status
TC-11	Skill Python with industry Analytics	Sneha Patel (4) listed before Rahul Verma (2)	PASS
TC-12	Misspelled query pythn	No mentor returned; the fuzzy part compares the query only with the first 80 characters of the skills text	OBSERVED
TC-13	Student sends a request to a mentor	HTTP 201, status pending	PASS
TC-14	Second request while one is pending	HTTP 409, "Request already pending."	PASS
TC-15	Mentor accepts a request	HTTP 200, status accepted; one student notification added	PASS
TC-16	Update with an invalid status	HTTP 400, "Invalid status"	PASS
TC-17	Update an unknown request	HTTP 404, "Request not found"	PASS
TC-18	Mentor publishes an opportunity	HTTP 201, post stored	PASS
TC-19	Create an event with capacity "40"	HTTP 201, capacity stored as the number 40	PASS
TC-20	Unknown API route	HTTP 404, "API route not found"	PASS
TC-21	Static request using ../ to reach server.js	HTTP 404, file not served	PASS
TC-22	Run all 34 algorithms with demonstration input	34 of 34 returned HTTP 200 with the expected result	PASS

5.4 Algorithm Output Validation

Each of the 34 algorithms was run through the /api/algorithms/run endpoint with its built-in input, and the output was compared with a hand calculation or a known textbook result. The outputs are listed in Table 5.3. For example, the suffix array and LCP array of banana are the standard values, the Needleman–Wunsch score of MENTOR and MENTEE is 4 matches × 2 − 2 mismatches = 6, the maximum flow equals the capacity of the cut (7 + 4 = 11), and the 2-approximation of vertex cover returns all six vertices of the 6-cycle, which is within the guaranteed factor of 2 of the optimal cover of size 3. The reservoir sample is random, so the value shown is one possible output.

Table 5.3 Output of the Algorithms for the Demonstration Inputs

Algorithm	Demonstration Input	Output
Naive search	text "alumni mentorship and alumni careers", pattern "alumni"	[0, 22]
KMP	"alumni connect platform connects alumni", "alumni"	[0, 33]
Z-function	"career mentorship alumni career", "career"	[0, 25]
Rabin–Karp	"Java DSA JavaScript DSA", "DSA"	[5, 20]
Aho–Corasick	"java python data science web development"; java, python, data	java at 0, python at 5, data at 12
Suffix array + LCP	"banana"	SA [5,3,1,0,4,2]; LCP [1,3,0,0,2]
Suffix automaton	"alumni"	7 states, 21 distinct substrings
Levenshtein	mentorship, mentors	3
Damerau–Levenshtein	alumni, almuni	1
Needleman–Wunsch	MENTOR, MENTEE	6
Smith–Waterman	CAREER, CARE	12
Matrix-chain	dimensions 10, 20, 30, 40, 30	30000
Optimal BST	keys A–D, frequencies 3, 2, 6, 4	26
Bitmask TSP	4 cities	80
Subset sum	values 3, 4, 7, 2, 9; target 13	true
0/1 Knapsack	weights 2, 3, 4, 5; values 3, 4, 5, 8; capacity 8	12
Hamiltonian path	4-vertex graph	[0, 1, 2, 3]
Tree/Subset DP	values 3, 4, 7, 2, 9; target 13	true
Tree diameter	4-vertex tree	3
SOS DP	[1, 2, 3, 4]	[1, 3, 4, 10]
Ford–Fulkerson	4-vertex network, source 0, sink 3	11
Min-cut	same network	source side {0,1,2}, sink side {3}
Edmonds–Karp	same network	11
Dinic	same network	11
Bipartite matching	3 students, 3 mentors (CSE–Java, ECE–IoT, IT–AI)	3 pairs matched
Min-cost max-flow	4-vertex graph with costs	flow 2, cost 11
Vertex cover 2-approx.	6-cycle	[0, 1, 2, 3, 4, 5]
Randomised Quicksort	[9, 3, 7, 1, 8, 2, 6, 5, 4]	[1, 2, 3, 4, 5, 6, 7, 8, 9]

Algorithm	Demonstration Input	Output
Miller–Rabin	104729	true (prime)
Universal-style hash	"alumni-connect"	63
Reservoir sampling	seven skills, k = 3	["Java", "DSA", "Cloud"] (random)
Parallel reduce	[4, 7, 2, 8, 3, 6]	30
Parallel prefix sum	[1, 2, 3, 4, 5]	[1, 3, 6, 10, 15]
Parallel sort	[9, 1, 5, 2, 8, 3]	[1, 2, 3, 5, 8, 9]

5.5 Search Performance Benchmark

The four single-pattern matchers were first checked for agreement. As shown in Table 5.4, all of them found the same number of matches for every pattern, which confirms that the implementations are consistent.

Table 5.4 Number of Matches Found in the 3,717-Character Corpus

Pattern	Naive	KMP	Z	Rabin–Karp
alumni	20	20	20	20
java	5	5	5	5
mentor	8	8	8	8

The average time per search is given in Table 5.5 and Fig. 5.2. On the small corpus, Naive and KMP are almost equal (about 0.035 ms and 0.036 ms), the Z algorithm is about twice as slow and Rabin–Karp about three to four times slower. On the larger corpus, KMP (2.19 ms) is slightly faster than Naive (2.23 ms), while Z (4.86 ms) and Rabin–Karp (6.32 ms) remain slower. Aho–Corasick is a multi-pattern algorithm and was therefore not included in this comparison; it found 33 occurrences of the three patterns together.

Table 5.5 Local Benchmark Results (Average Time per Search in ms)

Algorithm	alumni (small)	java (small)	mentor (small)	Average small	Average large
Naive	0.036743	0.032987	0.035126	0.034952	2.228620
KMP	0.038610	0.035550	0.034646	0.036269	2.185278
Z Algorithm	0.074350	0.063341	0.074214	0.070635	4.859558
Rabin–Karp	0.136821	0.114868	0.135609	0.129099	6.320067

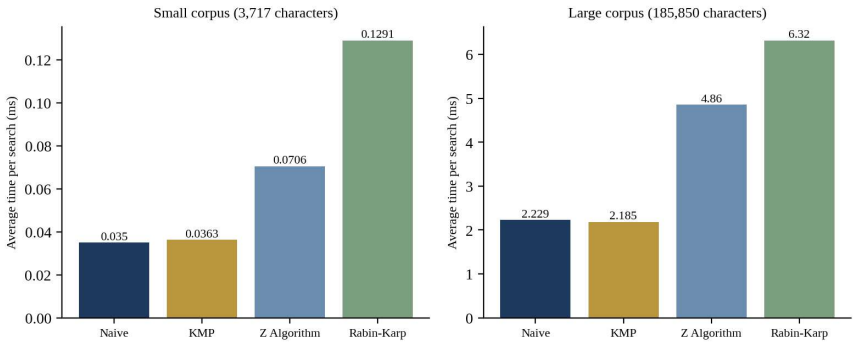


Fig. 5.2 Local Benchmark: Average Search Time of the Exact Matchers

5.6 User Interface Results

The user interface was exercised for both roles. Fig. 5.3 shows the Find Mentors page after the query java: only Priya Sharma is returned, with her skills as tags and the Request Mentorship button for the student. Fig. 5.4 shows the query python, which returns the two mentors whose profiles contain this word.

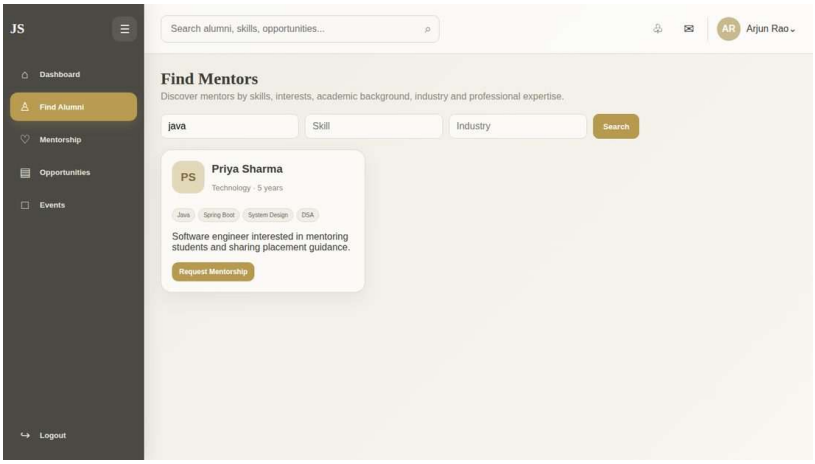


Fig. 5.3 Find Mentors: Search Result for the Query "java"

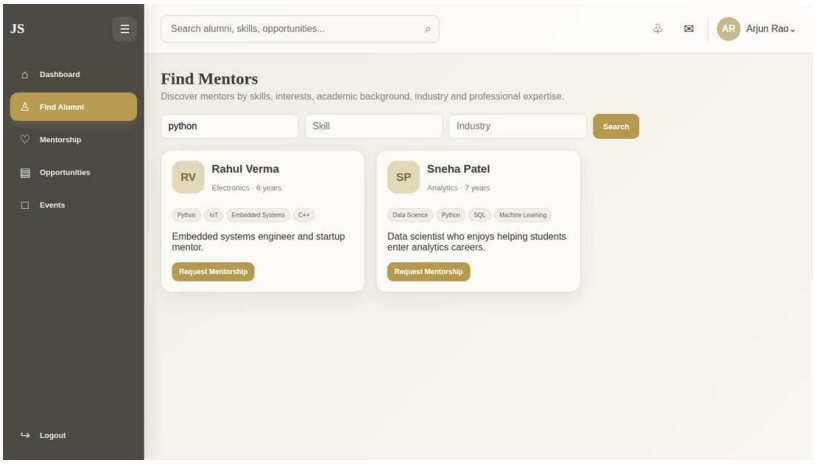


Fig. 5.4 Find Mentors: Search Result for the Query "python"

Fig. 5.5 shows the Mentorship page of the student, with one accepted request (Priya Sharma, placement preparation) and one pending request (Sneha Patel, data science roadmap). Fig. 5.6 and Fig. 5.7 show the Opportunities and Events pages with the demonstration posts and events.

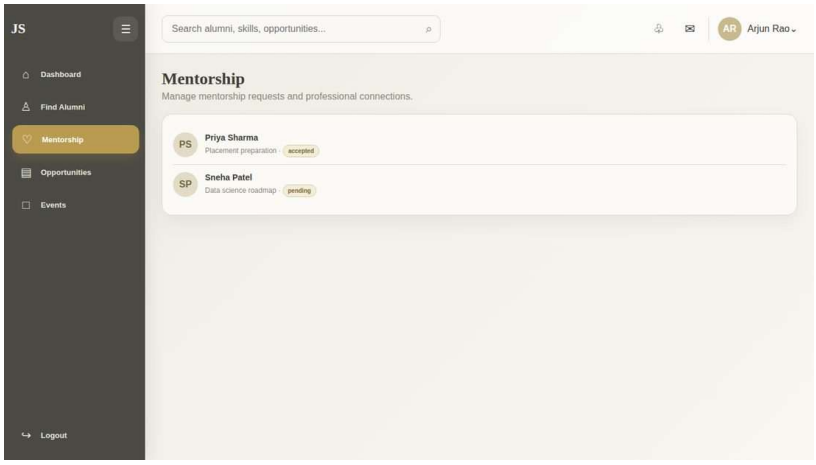


Fig. 5.5 Mentorship Page of the Student

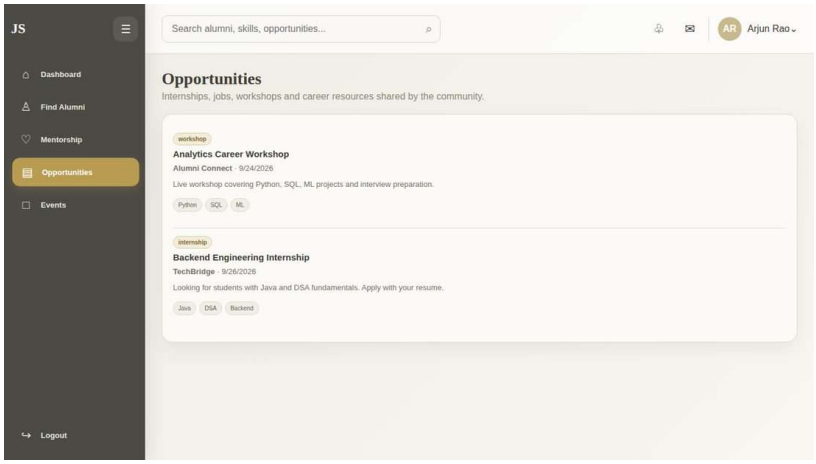


Fig. 5.6 Opportunities Page

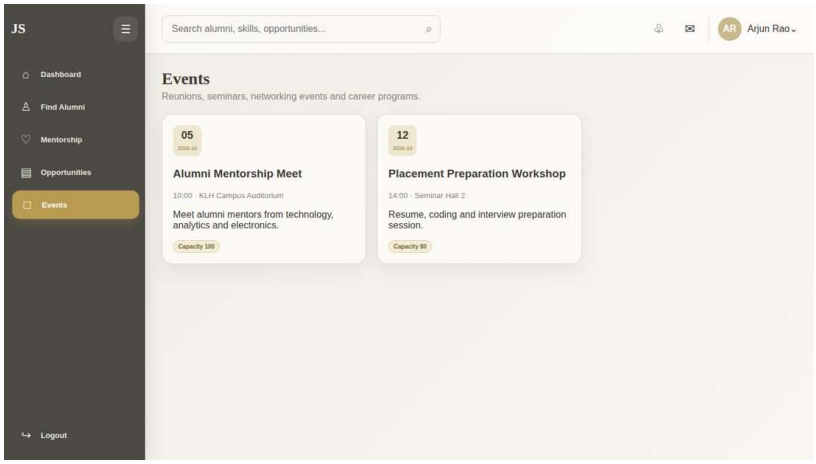


Fig. 5.7 Events Page

For the mentor role, the dashboard (Fig. 5.8) offers the Share an Opportunity shortcut and shows the mentor network, while the Mentorship page (Fig. 5.9) lists the request of the student. A request in the pending state would additionally show the Accept and Reject buttons.

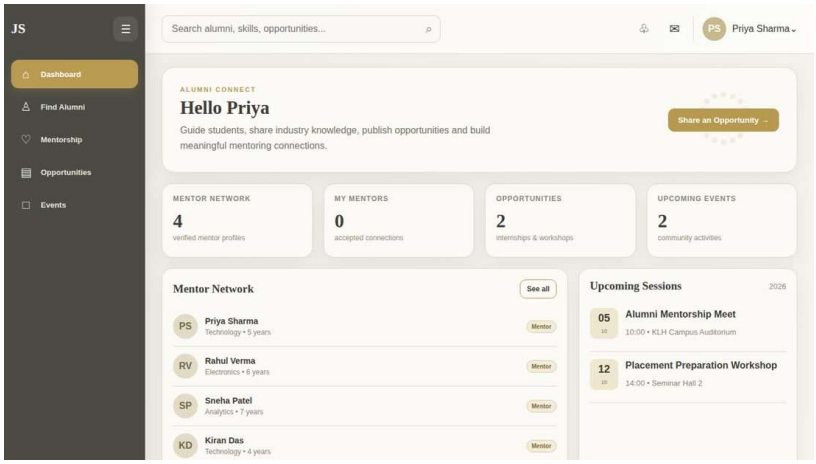


Fig. 5.8 Mentor Dashboard

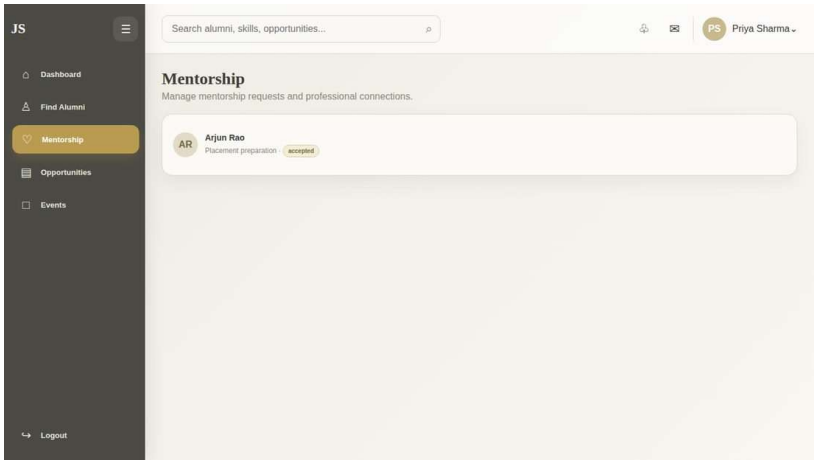


Fig. 5.9 Mentorship Page of the Mentor (Accepted Request)

5.7 Discussion of Results

The functional tests show that the main features of Alumni Connect behave as designed: authentication distinguishes wrong credentials from a wrong role, registration validates its input, mentor search returns ranked results, the mentorship workflow prevents duplicate pending requests and creates notifications, and invalid inputs return clear JSON errors. All 34 algorithms return correct results for their demonstration inputs.

The benchmark shows that the choice of algorithm affects the running time even when the results are identical. In this environment Naive and KMP are close to each other, which is expected for short patterns and natural text where Naive rarely needs many comparisons per alignment. The Z algorithm has to build a concatenated string and a Z-array, and Rabin–Karp performs modular arithmetic for every window, which probably explains their larger times on these inputs. The result also reminds us that asymptotic complexity must be read together with input size, pattern characteristics and implementation overhead.

The tests also show that the fuzzy part of the mentor search is narrow: it helps only when the whole query is close to the beginning of the skills text. A misspelled single word such as pythn does not match. This behaviour is a direct consequence of comparing the whole query with a prefix of the skills text and it is described as a limitation in Chapter 6.

6. CHALLENGES & LIMITATIONS

6.1 Issues Faced During Development

Storing data without a database. The project had to keep users, requests, posts, events and notifications without a database server. Plain text files are easy to inspect, but a damaged line or a partial write could make a whole file unreadable, and every update needs the complete file to be rewritten.

Role naming. The interface speaks of Student and Mentor, while the stored records and the original back end use the role alumni (and admin). Login, registration and the dashboard therefore had to translate between the two names without breaking existing records.

Combining exact and approximate matching. A ranking based only on substring tests ignores small differences in typing, while a pure edit-distance comparison of a short query with a long profile text is meaningless. A way had to be found to give each kind of evidence a suitable weight.

Keeping a large algorithm collection usable. The 34 algorithms differ in their inputs and outputs. They had to be reachable from one place without crowding the user interface, which led to the removal of the Algorithm Lab page from the visible menu.

Showing user-entered text safely. Names, topics, post contents and tags come from the users and are inserted into HTML templates, so unescaped text could break the page or inject markup.

Parallel algorithms in a single-threaded runtime. Node.js runs the server code in one thread, so parallel reduce, prefix sum and sort could only be demonstrated as sequential simulations.

6.2 Solutions Applied

- Each line of a data file is an independent JSON record, and readRecords() converts an unreadable line into a BAD record, so one damaged line does not stop the server.
- The login and registration handlers map the Mentor option to the stored role alumni, and the README documents this compatibility choice.

- The score of a mentor is built from three separate parts (whole-query match, token matches and a Levenshtein bonus), and the edit distance is computed only against the first 80 characters of the skills text to keep the comparison meaningful.
- All algorithms are placed behind the single runAlgorithm() dispatcher and the /api/algorithms endpoints, with a demonstration input for each, so that they can be tested without a visible page.
- The esc() function escapes the characters &, <, >, " and a single quote before any stored value is inserted into the page.
- The server checks for a pending duplicate before creating a mentorship request, and it validates the status value before an update.
- The static file handler normalises the requested path and serves only files inside the public folder.
- The parallel algorithms are clearly labelled as simulations in the algorithm list, so the report and the interface do not overstate them.

6.3 Limitations of the System

- **Plain-text storage.** The data is in text files and every update rewrites a complete file. Two simultaneous updates could overwrite each other and the approach does not scale to large data.
- **Passwords and sessions.** Passwords are stored and compared as plain text, and the browser keeps only the user record in localStorage. There is no token, so the API trusts the identifiers sent by the client.
- **No server-side role checks.** The roles are enforced by the interface. The POST endpoints for posts and events do not verify the role of the caller, and the administrator account cannot log in through the Student and Mentor login screen.
- **Narrow fuzzy matching.** The Levenshtein bonus compares the whole query with the first 80 characters of the skills text, so a misspelled single word (for example pythn, Fig. 6.1) returns no mentor.
- **Algorithms not wired to the interface.** Only the Levenshtein distance is used by a visible feature. The other algorithms are available through the API

but have no page, and the Ford–Fulkerson and Edmonds–Karp entries share one implementation.

- **Dashboard statistics.** The My Mentors tile counts requests in which the user is the student, so it shows 0 for a mentor who has accepted mentees.
- **Notifications.** Notifications are addressed to a role and not to a person, and there is no read status. The message and notification icons in the top bar are not connected to a messaging feature.
- **Small dataset and no automated tests.** The demonstration data has six users, and the verification in this report was done by manual and scripted API calls, not by an automated test suite.
- **Parallel algorithms are simulations** and the suffix-array construction uses a comparison sort, which is slower than the linear-time constructions.

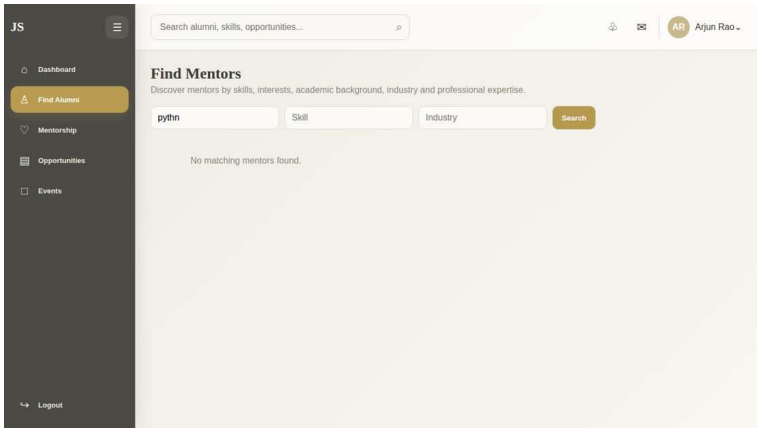


Fig. 6.1 No Result for the Misspelled Query "pythn"

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

Alumni Connect shows that a useful alumni–student mentoring platform can be built with transparent tools: a Node.js server that uses only built-in modules, text files as storage and a browser interface written in plain JavaScript. The platform provides role-based login and registration, a dashboard, a ranked mentor search, a complete mentorship workflow with notifications, an opportunities board and an events page, through fifteen documented API endpoints.

The project also meets its DSA-3 objective. Thirty-four algorithms from six categories (string matching, dynamic programming, network flow, approximation, randomised algorithms and parallel-style simulations) were implemented by hand and verified: all of them returned the expected result for their demonstration inputs, and the four exact string-matching algorithms found identical matches on the project text. The Levenshtein distance is part of the mentor-search score, which connects a classical dynamic-programming algorithm to a real feature of the platform.

The functional tests passed for all designed behaviours, and the benchmark showed that Naive and KMP were fastest on the tested text while Z and Rabin–Karp were slower, a result that depends on input size and environment. The tests also revealed limitations, mainly plain-text storage, plain-text passwords, missing server-side role checks and a narrow fuzzy match, which define the direction of future work.

7.2 Future Work

- Replace the text files with a database (for example SQLite or MongoDB) and add transactions, so that concurrent updates are safe.
- Store passwords as salted hashes and replace the localStorage user record by signed session tokens with server-side role checks on every endpoint.
- Improve the fuzzy search by applying edit distance word by word to each skill and interest, by using Damerau–Levenshtein to tolerate transposed letters, and by using Aho–Corasick to search many skills in one pass.

- Add autocomplete with a trie and a ranking that also uses TF-IDF weights, department and batch.
- Connect the flow-based algorithms to the interface, for example automatic student-to-mentor assignment with bipartite matching or min-cost max-flow, so that mentor capacity and student preferences are respected.
- Add messaging, per-user notifications with a read status, e-mail alerts, event registration within the stated capacity and a working administrator dashboard.
- Add resume and profile-photo upload, pagination of long lists and an analytics page for the number of requests and accepted mentorships.
- Add automated unit and integration tests for every algorithm and endpoint, and benchmark the algorithms on larger generated datasets.

7.3 Skills Learned During Development

- Hand implementation and comparison of string-matching algorithms (Naive, KMP, Z, Rabin–Karp, Aho–Corasick) and of suffix structures.
- Dynamic-programming formulation, table design and recurrence writing for edit distance, alignment, knapsack, subset sum, matrix-chain, optimal BST and bitmask TSP.
- Network-flow modelling with residual graphs, level graphs and bipartite matching, and the use of an approximation algorithm and randomised algorithms.
- Building a REST-style server with the Node.js http module: routing, JSON parsing, status codes, CORS headers, static file serving and safe path handling.
- Designing a role-based single-page interface with vanilla JavaScript, fetch(), state objects, modal forms and HTML escaping.
- Designing file-based persistence in JSON-lines format and understanding its limits compared with a database.
- Testing an API with scripted calls, designing a benchmark with warm-up runs and interpreting the difference between asymptotic complexity and measured time.

- Technical documentation: writing pseudocode, preparing figures and tables, and working as a team on development, testing and reporting.

REFERENCES

[1] Node.js Foundation, "Node.js v18 documentation: HTTP, File System, Path and Crypto modules," [Online]. Available: https://nodejs.org/docs/latest-v18.x/api/

[2] "Alumni Connect – DSA-3 Project," README.md and source code (server.js, algorithms.js, public/ and data/), supplied project repository.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[4] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.

[5] D. E. Knuth, J. H. Morris, Jr. and V. R. Pratt, "Fast pattern matching in strings," SIAM J. Comput., vol. 6, no. 2, pp. 323–350, 1977.

[6] R. M. Karp and M. O. Rabin, "Efficient randomized pattern-matching algorithms," IBM J. Res. Develop., vol. 31, no. 2, pp. 249–260, 1987.

[7] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," Commun. ACM, vol. 18, no. 6, pp. 333–340, 1975.

[8] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[9] R. A. Wagner and M. J. Fischer, "The string-to-string correction problem," J. ACM, vol. 21, no. 1, pp. 168–173, 1974.

[10] S. B. Needleman and C. D. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," J. Mol. Biol., vol. 48, no. 3, pp. 443–453, 1970.

[11] T. F. Smith and M. S. Waterman, "Identification of common molecular subsequences," J. Mol. Biol., vol. 147, no. 1, pp. 195–197, 1981.

[12] L. R. Ford, Jr. and D. R. Fulkerson, "Maximal flow through a network," Canadian J. Math., vol. 8, pp. 399–404, 1956.

[13] J. Edmonds and R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems," J. ACM, vol. 19, no. 2, pp. 248–264, 1972.

[14] E. A. Dinic, "Algorithm for solution of a problem of maximum flow in a network with power estimation," Soviet Math. Doklady, vol. 11, pp. 1277–1280, 1970.

[15] C. A. R. Hoare, "Quicksort," Computer J., vol. 5, no. 1, pp. 10–16, 1962.

[16] M. O. Rabin, "Probabilistic algorithm for testing primality," J. Number Theory, vol. 12, no. 1, pp. 128–138, 1980.

[17] J. S. Vitter, "Random sampling with a reservoir," ACM Trans. Math. Softw., vol. 11, no. 1, pp. 37–57, 1985.

[18] U. Manber and G. Myers, "Suffix arrays: A new method for on-line string searches," SIAM J. Comput., vol. 22, no. 5, pp. 935–948, 1993.

[19] T. Kasai, G. Lee, H. Arimura, S. Arikawa and K. Park, "Linear-time longest-common-prefix computation in suffix arrays and its applications," in Proc. CPM 2001, LNCS vol. 2089. Berlin, Germany: Springer, 2001, pp. 181–192.

[20] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," J. SIAM, vol. 10, no. 1, pp. 196–210, 1962.

[21] A. Blumer, J. Blumer, D. Haussler, A. Ehrenfeucht, M. T. Chen and J. Seiferas, "The smallest automaton recognizing the subwords of a text," Theor. Comput. Sci., vol. 40, pp. 31–55, 1985.

[22] Mozilla Developer Network, "Fetch API," [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API